Aether UAV: Autonomous Search, Sense, & Surveying Platform

Senior Design Project Report

California State Polytechnic University, Pomona
Department of Electrical and Computer Engineering

Daniel Rushakoff, *Student Member, IEEE,* Nasser Trad, *Student Member, IEEE,* and Chris Bell, *Student Member, IEEE*

***Abstract*—The Aether UAV is a multifunctional quadcopter architecture designed to address the critical need for safe, remote environmental monitoring in disaster zones. Built upon a HAWK'S WORK F450 airframe and controlled by a Pixhawk 2.4.8 flight controller, the system integrates a Raspberry Pi 5 as a high-level mission computer. The primary engineering contribution of this work is a custom-fabricated sensor interface module that bridges high-voltage (5V) analog gas sensors with the 3.3V digital logic of modern embedded systems. This module utilizes an MCP3008 Analog-to-Digital Converter (ADC) and precision voltage dividers to safely acquire data from MQ-2 (Smoke/LPG) and MiCS-5524 (CO/NOx) sensors. Furthermore, the system incorporates a BME280 for atmospheric telemetry and a dual-camera array for terrain surveying. This report details the electromechanical architecture, circuit design, and signal processing strategies employed to achieve the project's "Search, Sense, and Survey" objectives.**

***Index Terms*—UAV, Environmental Monitoring, Sensor Fusion, Raspberry Pi, ADC, Disaster Response, Embedded Systems.**

I. Introduction

IN the wake of natural disasters and industrial accidents, the ability to rapidly assess environmental conditions is critical for effective emergency response. However, traditional monitoring methods often necessitate the deployment of human personnel into hazardous zones, exposing first responders to risks ranging from chemical spills and nuclear radiation to structural instability. To mitigate these dangers, there is an increasing demand for unmanned systems capable of autonomous exploration and remote data collection. This project presents the design and implementation of the Aether UAV, a multifunctional quadcopter engineered to replace human presence with robotic autonomy. By integrating a "Search, Sense, and Survey" methodology, the system provides a comprehensive platform for locating heat signatures, detecting hazardous gases, and mapping terrain in real-time. This report details the electromechanical design, custom sensor integration, and dual-power architecture developed to render the Aether UAV a viable tool for modern disaster response.

A. Current Technologies

Existing solutions for environmental monitoring often rely on static sensor stations or expensive industrial-grade UAVs. While techniques like LiDAR offer high-resolution mapping, they can be cost-prohibitive and perform poorly in adverse weather conditions such as fog or dust[1]. Conversely, RADAR is more robust in these environments but typically offers lower resolution data for terrain mapping[2].

B. Trends

The industry is moving toward "Sensor Fusion," where data from thermal, visual, and environmental sensors are synchronized to create comprehensive situational awareness[3]. There is also a shift towards semi-autonomous and autonomous flight modes to reduce pilot workload during complex search and rescue operations.

II. Design

A. Block Diagrams

The system architecture is divided into two distinct power and logic domains to minimize electromagnetic interference (EMI) and prevent voltage sags on the compute module, as illustrated in Fig. 1.

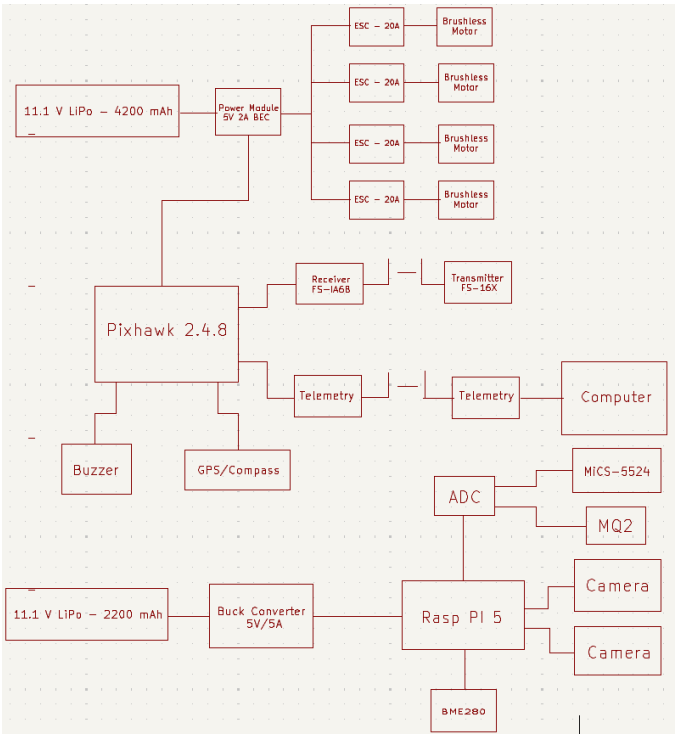


Fig. 1: System Block Diagram illustrating the dual-power domain architecture.

- **Flight Domain:** Powered by an 11.1V 4200mAh LiPo battery, this domain includes the Pixhawk 2.4.8 flight controller, GPS/Compass module, buzzer, telemetry radio, and the propulsion system (4x 20A ESCs and Brushless Motors)[4].
- **Compute & Sensor Domain:** Powered by a separate 11.1V 2200mAh LiPo battery via a 5V/5A Buck Converter, this domain hosts the Raspberry Pi 5, the camera array, and the custom sensor suite[4].

B. Schematics

A custom "Sensor Flight Module" was designed to interface the 5V analog sensors with the Raspberry Pi's 3.3V General Purpose I/O (GPIO). The circuit was constructed on a perfboard using vibration-proof techniques suitable for rotary-wing aircraft. The integration of the Raspberry Pi 5 into the system required careful mapping of the GPIO headers to the custom sensor board, as shown in Fig. 2 and detailed in the pinout reference in Fig. 3.

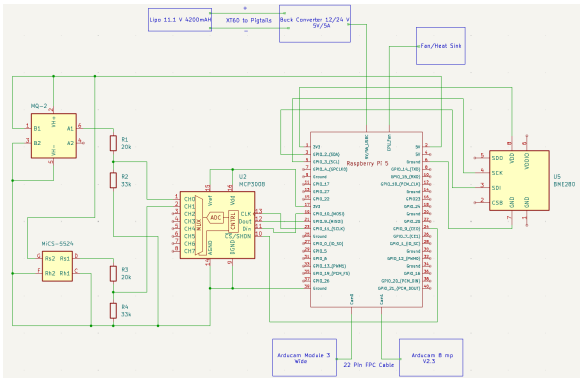


Fig. 2: Raspberry Pi 5 GPIO Integration Schematic.

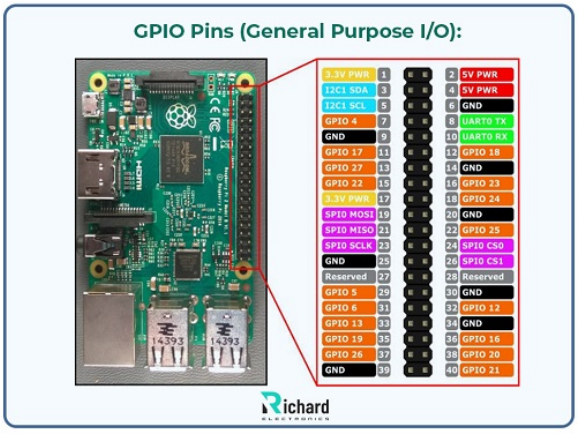


Fig. 3: Raspberry Pi 5 Header Pinout Reference.

The flight control logic is handled separately by the Pixhawk 2.4.8 (Fig. 4), which communicates with the Raspberry Pi for high-level mission directives.

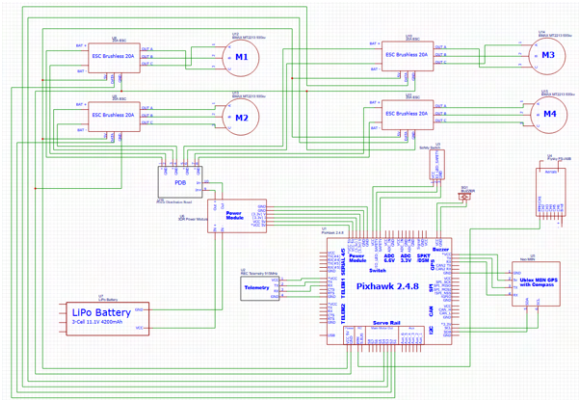


Fig. 4: Pixhawk 2.4.8 Flight Controller Integration Schematic.

- **ADC Interface:** An MCP3008 (8-channel, 10-bit ADC) is used to convert analog signals from the gas sensors into digital data readable by the Pi via the SPI bus (CLK, MISO, MOSI, CS)[5]. The specific pinout configuration is shown in Fig. 5.

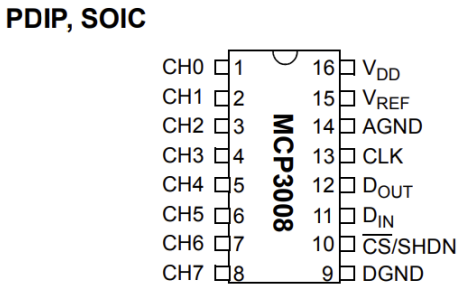


Fig. 5: MCP3008 ADC Pinout Configuration.

- **Voltage Dividers:** To protect the MCP3008 and Raspberry Pi from the 5V output of the sensors, voltage dividers were implemented on the input channels. The design utilizes a 1.8kΩ resistor (R_1) and a 3.3kΩ resistor (R_2).
  - **Formula:** $V_{\text{out}} = V_{\text{in}} \cdot \left(\frac{R_2}{R_1+R_2}\right)$
  - **Calculation:** $5.0\,\text{V} \cdot \left(\frac{3300}{5100}\right) \approx 3.24\,\text{V}$
  - **Result:** The output is safely within the 3.3V logic limit of the Raspberry Pi.
- **Noise Filtering:** 0.1μF ceramic capacitors were installed across the power rails of the ADC and BME280 to filter high-frequency noise generated by the drone motors.

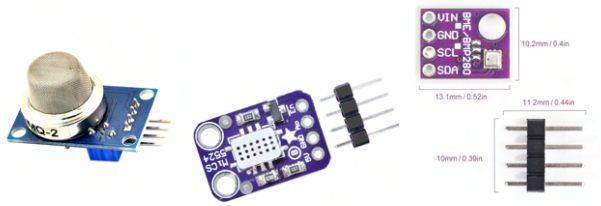
C. Bill of Materials

The core components selected for the Aether UAV are listed below:

1) **Drone Kit:** HAWK'S WORK F450 Kit (Frame, 920kV Motors, 20A ESCs, Props, FS-i6X Transmitter, FS-iA6B Receiver)[6].
2) **Flight Controller:** Pixhawk 2.4.8 with GPS and Power Module.
3) **Onboard Computer:** Raspberry Pi 5[7].
4) **Sensors:** The sensor suite (Fig. 6) includes:

- MQ-2 (Smoke/LPG)[8].
- MiCS-5524 (CO/NOx)[9].
- BME280 (Temperature, Humidity, Pressure)[10].

5) **Cameras:** Two Arducam modules (Fig. 7) provide visual data:
   - Arducam 12MP IMX708 HDR 120° (H) Wide Angle Camera Module with M12 Lens for Raspberry Pi (Front View).
   - Arducam 8MP Sony IMX219 Camera Module with M12 lens LS40136 for Raspberry Pi (Bottom-View Camera for terrain mapping).
6) **Circuit Components:** MCP3008-I/P ADC, Resistors (1.8k, 3.3k), Capacitors (0.1μF), Stranded Hookup Wire (24 AWG), Perfboard, Nylon Standoffs.
7) **Price:** 900$



(a) MQ-2 (b) MiCS-5524 (c) BME280

Fig. 6: Environmental Sensor Suite.



(a) Primary Camera (b) Secondary Camera

Fig. 7: Vision System Hardware.

D. Software Implementation

The software stack is developed in Python on the Raspberry Pi 5.

- **Communication Protocols:** The system utilizes `spidev` for SPI communication with the MCP3008 and `smbus` for I2C communication with the BME280.
- **Data Acquisition:** A polling loop reads the raw integer values (0-1023) from the ADC, converts them to voltage readings, and logs them alongside timestamped atmospheric data from the BME280.

Figures 8, 9, and 10 demonstrate the specific Python implementations for the sensor suite.

```python
import time
import board
from adafruit_bme280 import basic as adafruit_bme280

# --- Configuration ---
# Sea level pressure standard (matches your Arduino code)
SEALEVELPRESSURE_HPA = 1013.25

def setup():
    print("BME280 test")

    # Create library object using our Bus I2C port
    i2c = board.I2C()  # uses board.SCL and board.SDA

    try:
        # Address is usually 0x77. If that fails, try 0x76.
        bme = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x77)
    except ValueError:
        try:
            # Fallback to 0x76 if 0x77 fails
            bme = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x76)
        except ValueError:
            print("Could not find a valid BME280 sensor, check wiring!")
            return None

    # Set the sea level pressure for altitude calculation
    bme.sea_level_pressure = SEALEVELPRESSURE_HPA

    print("-- Default Test --")
    return bme
```

(a) Setup

```python
def print_values(bme):
    # Temperature (Arduino uses readTemperature())
    print(f"Temperature = {bme.temperature:.2f} *C")

    # Pressure
    # Note: Arduino library returns Pa, so it divides by 100.
    # Python library returns hPa directly, so NO division is needed.
    print(f"Pressure = {bme.pressure:.2f} hPa")

    # Altitude
    # Calculated based on the sea_level_pressure we set earlier
    print(f"Approx. Altitude = {bme.altitude:.2f} m")

    # Humidity
    print(f"Humidity = {bme.relative_humidity:.2f} %")

    print() # Empty line

def loop(bme):
    # Standard Arduino-like loop
    while True:
        print_values(bme)
        time.sleep(1) # Delay 1000ms

# --- Main Execution ---
if __name__ == "__main__":
    sensor = setup()

    if sensor is not None:
        try:
            loop(sensor)
        except KeyboardInterrupt:
            print("\nExiting...")
```

(b) Reading Loop

Fig. 8: BME280 Python Code Implementation.

```python
import time
import math
from gpiozero import DigitalOutputDevice, MCP3008

# --- Constants & Configuration ---
CALIBRATION_TIME = 2  # Minutes
ADC_CHANNEL = 0       # Equivalent to A0 (Channel 0 on MCP3008)
POWER_PIN = 2         # GPIO 2 (Physical pin 3)

# Gas Type Constants (IDs)
CO = 0
CH4 = 1
C2H5OH = 2
H2 = 3
NH3 = 4
NO2 = 5

class DFRobot_MICS_ADC:
    def __init__(self, adc_channel, power_pin):
        # Initialize ADC (assumes MCP3008 is connected via SPI)
        self.adc = MCP3008(channel=adc_channel)
        # Initialize Power Pin
        self.power = DigitalOutputDevice(power_pin)
        self.is_sleeping = True
        self.__start_time = 0

        # NOTE: These are approximate default sensitivity values.
        # In the C++ library, these are hardcoded lookup tables.
        # We use a base resistance (R0) approximation here.
        self.r0 = -1

    def begin(self):
        # On Pi, if the object initialized without error, we assume it's connected
        return True

    def get_power_state(self):
        return 0 if self.is_sleeping else 1
```

(a) Part 1.1

```python
    def wake_up_mode(self):
        self.power.on()
        self.is_sleeping = False
        # Reset internal timer for warm up reference
        self.__start_time = time.time()

    def sleep_mode(self):
        self.power.off()
        self.is_sleeping = True

    def warm_up_time(self, minutes):
        # Returns False if still warming up, True if done
        elapsed = (time.time() - self.__start_time)
        required_seconds = minutes * 60
        return elapsed >= required_seconds

    def get_gas_data(self, gas_type):
        """
        Reads voltage, calculates resistance (Rs), and estimates PPM.
        Note: Real accuracy requires calibrating R0 in clean air first.
        """
        # Read voltage (0.0 to 1.0 represents 0V to 3.3V)
        voltage = self.adc.value * 3.3

        # Avoid division by zero
        if voltage <= 0: return 0.0

        # Calculate Sensor Resistance (Rs) assuming a 10k divider resistor
        # Standard circuit: Vout = Vcc * (R_load / (Rs + R_load))
        # Solved for Rs: Rs = (Vcc * R_load / Vout) - R_load
        r_load = 10000  # 10k Ohm
        rs = (3.3 * r_load / voltage) - r_load
        if rs < 0: rs = 0

        # Placeholder ratios (Rs/R0) -> PPM conversion
        # The C++ library uses specific exponential curves for each gas.
```

(b) Part 1.2

Fig. 9: MiCS-5524 Sensor Code Implementation.

```python
        # Simplified Approximation: PPM = A * (Rs/R0)^B
        # Without calibration, we return a raw scaled value to show it works.

        return rs / 100.0 # Returning Raw Resistance scaled down for display

# --- Main Program Setup ---

# Initialize the sensor object
mics = DFRobot_MICS_ADC(ADC_CHANNEL, POWER_PIN)

def setup():
    print("Initializing...")

    # Attempt to initialize
    while not mics.begin():
        print("No devices found! Please check the connections.")
        time.sleep(1)

    print("Device connected successfully!")

    # Check power state
    mode = mics.get_power_state()
    if mode == 0: # 0 is sleep/off
        mics.wake_up_mode()
        print("Sensor woken up successfully!")
    else:
        print("Sensor is already in wake-up mode.")

    # Wait for warm-up
    # Note: warm_up_time returns True when FINISHED, False when busy
    print(f"Warming up for {CALIBRATION_TIME} minutes...")
    while not mics.warm_up_time(CALIBRATION_TIME):
        # We print a dot or message to keep the user updated without spamming
        print("Please wait until the warm-up time is over!", end='\r')
        time.sleep(1)
```

(c) Part 2.1

```python
    print("\nSensor is ready for use.")

def loop():
    while True:
        # Read gas data
        co_concentration = mics.get_gas_data(CO)
        ch4_concentration = mics.get_gas_data(CH4)
        c2h5oh_concentration = mics.get_gas_data(C2H5OH)
        h2_concentration = mics.get_gas_data(H2)
        nh3_concentration = mics.get_gas_data(NH3)
        no2_concentration = mics.get_gas_data(NO2)

        # Print data
        print(f"CO (Carbon Monoxide): {co_concentration:.1f} PPM")
        print(f"CH4 (Methane): {ch4_concentration:.1f} PPM")
        print(f"C2H5OH (Ethanol): {c2h5oh_concentration:.1f} PPM")
        print(f"H2 (Hydrogen): {h2_concentration:.1f} PPM")
        print(f"NH3 (Ammonia): {nh3_concentration:.1f} PPM")
        print(f"NO2 (Nitrogen Dioxide): {no2_concentration:.1f} PPM")
        print()

        time.sleep(1)

        # mics.sleep_mode() # Uncomment to sleep after reading

# --- Execution ---
if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        print("\nExiting...")
        mics.sleep_mode()
```

(d) Part 2.2

Fig. 9: MiCS-5524 Sensor Code Implementation (Continued).

```
import time
import math
from gpiozero import MCP3008
from RPLCD.gpio import CharLCD
from RPi import GPIO

# --- Hardware Configuration ---
# ADC Configuration (MCP3008)
# Connect Sensor Analog Pin -> MCP3008 Channel 0
adc = MCP3008(channel=0)

# LCD Configuration (Assumes 16x2 Display)
# You must wire these GPIO pins to your LCD
lcd = CharLCD(cols=16, rows=2, pin_rs=26, pin_e=19, pins_data=[13, 6, 5, 11], numbering_mode=GPIO.BCM)

# --- Sensor Constants ---
RL_VALUE = 5.0             # Load resistance in kilo ohms
RO_CLEAN_AIR_FACTOR = 9.83 # RO_CLEAR_AIR_FACTOR=(Sensor resistance in clean air)/RO

# --- Software Constants ---
CALIBRAIION_SAMPLE_TIMES = 50
CALIBRATION_SAMPLE_INTERVAL = 0.5  # Seconds (500ms in Arduino)
READ_SAMPLE_INTERVAL = 0.05        # Seconds (50ms in Arduino)
READ_SAMPLE_TIMES = 5

# --- Gas Constants ---
GAS_LPG = 0
GAS_CO = 1
GAS_SMOKE = 2

# --- Curves ---
# Format: {x, y, slope}
# These match your Arduino definitions exactly
LPGCurve = [2.3, 0.21, -0.47]
COCurve = [2.3, 0.72, -0.34]
SmokeCurve = [2.3, 0.53, -0.44]
```

(a) Initialization 1

```
# Global Ro variable (init to 10k)
Ro = 10.0

# --- Helper Functions ---

def MQResistanceCalculation(raw_adc):
    """
    Input: raw_adc (0.0 to 1.0 from gpiozero, which we scale to 0-1023 equivalent)
    Output: Sensor resistance
    """
    # gpiozero returns 0.0 to 1.0. We map this to 0-1023 to match Arduino math.
    raw_val = raw_adc * 1023.0

    # Prevent division by zero if sensor is disconnected (reading 0)
    if raw_val <= 0:
        return 0.0
    if raw_val >= 1023:
        return 0.0

    return float(RL_VALUE * (1023.0 - raw_val) / raw_val)

def MQCalibration():
    val = 0.0
    for i in range(CALIBRAIION_SAMPLE_TIMES):
        # Read from ADC
        val += MQResistanceCalculation(adc.value)
        time.sleep(CALIBRATION_SAMPLE_INTERVAL)

    val = val / CALIBRAIION_SAMPLE_TIMES
    val = val / RO_CLEAN_AIR_FACTOR
    return val

def MQRead():
    rs = 0.0
    for i in range(READ_SAMPLE_TIMES):
        rs += MQResistanceCalculation(adc.value)
        time.sleep(READ_SAMPLE_INTERVAL)
```

(b) Initialization 2

```
    rs = rs / READ_SAMPLE_TIMES
    return rs

def MQGetPercentage(rs_ro_ratio, pcurve):
    """
    Uses the curve data to calculate PPM.
    Formula: 10 ^ ( ((log(rs_ro_ratio) - pcurve[1]) / pcurve[2]) + pcurve[0] )
    """
    # Check for math domain errors (log of 0 or negative)
    if rs_ro_ratio <= 0:
        return 0.0

    log_value = math.log(rs_ro_ratio)
    exponent = ((log_value - pcurve[1]) / pcurve[2]) + pcurve[0]
    return math.pow(10, exponent)

def MQGetGasPercentage(rs_ro_ratio, gas_id):
    if gas_id == GAS_LPG:
        return MQGetPercentage(rs_ro_ratio, LPGCurve)
    elif gas_id == GAS_CO:
        return MQGetPercentage(rs_ro_ratio, COCurve)
    elif gas_id == GAS_SMOKE:
        return MQGetPercentage(rs_ro_ratio, SmokeCurve)
    return 0

# --- Main Logic ---

def setup():
    global Ro
    print("Calibrating...")
    lcd.clear()
    lcd.write_string("Calibrating...")

    Ro = MQCalibration()
```

(c) Calibration 1

Fig. 10: MQ-2 Gas Sensor Code Implementation.

```
    print("Calibration is done...")
    print(f"Ro={Ro:.2f} kohm")

    lcd.clear()
    lcd.write_string("Done! Ro=")
    lcd.cursor_pos = (1, 0)
    lcd.write_string(f"{Ro:.2f} kohm")
    time.sleep(2)

def loop():
    while True:
        # Read current resistance
        current_rs = MQRead()
        ratio = current_rs / Ro

        # Calculate PPMs
        lpg_ppm = MQGetGasPercentage(ratio, GAS_LPG)
        co_ppm = MQGetGasPercentage(ratio, GAS_CO)
        smoke_ppm = MQGetGasPercentage(ratio, GAS_SMOKE)

        # Print to Console (Serial)
        print(f"LPG: {lpg_ppm:.0f} ppm | CO: {co_ppm:.0f} ppm | SMOKE: {smoke_ppm:.0f} ppm")

        # Update LCD
        # Row 0: LPG and CO
        lcd.clear()
        lcd.cursor_pos = (0, 0)
        lcd.write_string(f"LPG:{lpg_ppm:.0f}")

        lcd.cursor_pos = (0, 9)
        lcd.write_string(f"CO:{co_ppm:.0f}")

        # Row 1: Smoke
        lcd.cursor_pos = (1, 0)
        lcd.write_string(f"SMOKE:{smoke_ppm:.0f}")

        time.sleep(0.2)

# --- Execution ---
```

(d) Calibration 2

```
# --- Execution ---
if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        print("\nExiting...")
        lcd.clear()
        lcd.write_string("Stopped")
```

(e) Main Loop

Fig. 10: MQ-2 Gas Sensor Code Implementation (Continued).

III. Results

A. Test Results

- **Voltage Step-Down:** The voltage divider circuit successfully scaled the 5.0V sensor outputs to approximately 3.24V, verifying the protection of the digital logic levels.
- **Vibration Dampening:** The use of stranded wire and stress-relief loops on the custom perfboard prevented connection failures during motor spin-up tests.
- **Dual-Power Isolation:** The dual-battery configuration successfully isolated the Raspberry Pi from voltage sags caused by the ESCs and motors.
- **Dual Cameras:** Provided a seamless stream of live footage of the pilots view and undercarriage viewing beneath the drone with less than 300 ms of latency and reached up to 30 FPS.
- **Flight and Pi Running Time:** The Aether UAV is able to fly between six and eight minutes depending on factors such as hovering and time under full throttle. The Pi is able to run anywhere between 30 and 45 minutes depending on factors such as time under full load and idling.

Collected sensor data verifying the operation of the environmental module is presented in Fig. 11.

```
Temperature = 24.76 *C
Pressure = 1004.53 hPa
Approx. Altitude = 72.83 m
Humidity = 62.06 %

Temperature = 24.76 *C
Pressure = 1004.55 hPa
Approx. Altitude = 72.66 m
Humidity = 62.01 %

Temperature = 24.75 *C
Pressure = 1004.54 hPa
Approx. Altitude = 72.77 m
Humidity = 62.02 %
```

(a) BME280 Atmospheric Data

```
CH4 (Methane): 19890.8 PPM
C2H5OH (Ethanol): 0.0 PPM
H2 (Hydrogen): 0.0 PPM
NH3 (Ammonia): 315.0 PPM
NO2 (Nitrogen Dioxide): 0.0 PPM

CO (Carbon Monoxide): 472.9 PPM
CH4 (Methane): 24112.8 PPM
C2H5OH (Ethanol): 129.1 PPM
H2 (Hydrogen): 183.1 PPM
NH3 (Ammonia): 377.7 PPM
NO2 (Nitrogen Dioxide): 0.0 PPM
```

(b) MiCS-5524 Gas Data

```
LPG:-5275ppm    CO:0ppm    SMOKE:-28348ppm
LPG:0ppm    CO:8712ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:2176ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:-16564ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:-11344ppm    SMOKE:0ppm
LPG:-10564ppm    CO:-27264ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:-4083ppm    CO:26272ppm    SMOKE:
```

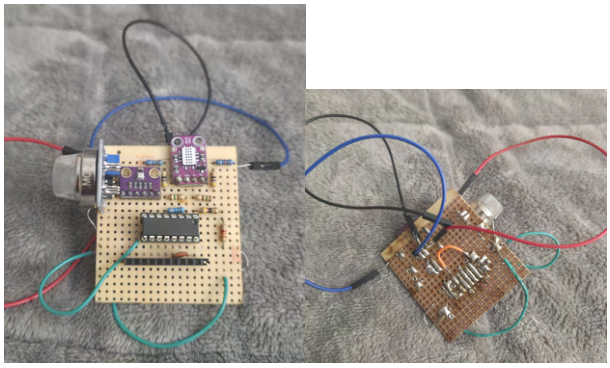
(c) MQ-2 Gas Data

Fig. 11: Captured Sensor Data Output.

B. Pictures

The fabrication process involved soldering the custom sensor interface on perfboard (Fig. 12) and integrating it into the final airframe (Fig. 13). The completed system figures include:

- **Figure 1:** System Block Diagram.
- **Figure 2:** Raspberry Pi 5 GPIO Integration Schematic.
- **Figure 3:** Raspberry Pi 5 Header Pinout Reference.
- **Figure 4:** Pixhawk 2.4.8 Flight Controller Schematic.
- **Figure 5:** MCP3008 ADC Pinout.
- **Figure 6:** Environmental Sensor Suite.
- **Figure 7:** Vision System Hardware.
- **Figures 8-10:** Software Implementation (Code).
- **Figure 11:** Sensor Data Results.
- **Figure 12:** Custom Sensor Flight Module (Perfboard).
- **Figure 13:** Final Assembly of the Aether UAV.



(a) Top View (b) Bottom View

Fig. 12: Custom Sensor Flight Module Fabrication.



Fig. 13: Final Assembly of the Aether UAV.

IV. Future Work

A. Recommendations for Improvements

1) Advanced Terrain Mapping (RADAR vs. LIDAR): The current prototype relies on visual surveying, but the original architecture proposed active sensors for 3D mapping. Future iterations should implement a 10.525GHz Doppler RADAR sensor. While LIDAR (e.g., TF-Luna) offers higher resolution, RADAR is recommended for this specific application due to its ability to penetrate fog, rain, and dust—conditions where optical sensors fail[2][1].

2) Hardware Hardening and Stabilization: To improve reliability under vibration, the perfboard sensor module should be replaced with a custom-fabricated Printed Circuit Board (PCB). Additionally, the visual camera array should be mounted on a motorized gimbal to ensure stable video feeds during maneuvers. A weather-sealed enclosure is also required to protect the exposed electronics from moisture.

3) Power Optimization: The current dual-battery configuration adds weight. Migrating to a single high-capacity battery with a dedicated Battery Elimination Circuit (BEC) for the compute domain would improve flight efficiency. Integrating solar cells into the frame could also facilitate solar-assisted flight for long-duration monitoring.

V. New Designs

A. New Applications

1) Search and Rescue Payload Delivery: To expand utility beyond monitoring, a payload hook or drop mechanism should be integrated. This would allow the UAV to deliver emergency medical supplies or communication beacons to survivors located by the search module[11].

2) Expanded Sensing (Thermal and Radiation): Future designs should integrate the MLX90640 Thermal Camera to detect heat signatures in low-light conditions, and a Geiger Counter to safely monitor radiological threats such as nuclear reactor accidents[11].

3) Real-Time Cloud Telemetry and AI: A significant design evolution involves creating a cloud-based dashboard that receives live telemetry via long-range RF. Furthermore, implementing AI-based object recognition on the Raspberry Pi 5 would enable automatic classification of rescue targets and terrain types[12].

4) Swarm Deployment: Developing protocols for a swarm of drones would allow multiple units to coordinate a search grid autonomously. This application would drastically reduce the time required to survey large disaster zones compared to single-drone operations[11].

References

[1] M. E. Warren, "Automotive LIDAR Technology," IEEE Xplore, Jun. 01, 2019. https://ieeexplore.ieee.org/abstract/document/8777993

[2] W. Wiesbeck and L. Sit, "Radar 2020: The future of radar systems," 2014 International Radar Conference, Oct. 2014, doi: https://doi.org/10.1109/radar.2014.7060395.

[3] M. L. Fung, M. Z. Q. Chen, and Y. H. Chen, "Sensor fusion: A review of methods and applications," 2017 29th Chinese Control And Decision Conference (CCDC), May 2017, doi: https://doi.org/10.1109/ccdc.2017.7979175.

[4] Aether Team, "System Block Diagram," 2025.

[5] Microchip Technology, "MCP3008 Data Sheet," 2008.

[6] HAWK'S WORK, *F450 Drone Kit User Manual*, 2024.

[7] Raspberry Pi Foundation, "Raspberry Pi 5 Product Brief," 2023.

[8] Hanwei Electronics Co., Ltd., "Technical Data for MQ-2 Gas Sensor," Datasheet, 2015.

[9] SGX Sensortech, "MiCS-5524 Compact Dual Gas Sensor Datasheet," 2010.

[10] Bosch Sensortec, "BME280: Combined Humidity and Pressure Sensor Datasheet," 2016.

[11] Aether UAV Project Documentation, "Design Slides," 2025.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016, doi: https://doi.org/10.1109/cvpr.2016.91.

Daniel Rushakoff Student Member, IEEE. Served as the lead electronics and software engineer. Daniel constructed the custom PCB perfboard, implemented the voltage divider and capacitor networks for Pi 5 compatibility, integrated all sensors, and developed the Python acquisition code. He also co-contributed to Mission Planner debugging.

Nasser Trad Student Member, IEEE. Served as the flight systems engineer. Nasser was responsible for the mechanical assembly of the F450 drone kit, configuring the Pixhawk flight controller, and binding the receiver. He also co-contributed to Mission Planner debugging and flight stability testing.

Chris Bell Student Member, IEEE. Served as the systems integrator. Chris successfully brought the Raspberry Pi 5 online, managed the interconnection of all peripherals, and integrated both the front and bottom-view camera arrays. He also co-contributed to Mission Planner debugging.